APEDP Phase 0: Technical Hurdles & Google Colab Implementation Blueprint

Document Version: 1.0.0
Target Platform: Autonomous Physics Engineering Discovery Platform (APEDP)
Execution Environment: Google Colab / Local Jupyter Notebook
Language/Runtime: Python 3.10+ (SymPy, Pint, NumPy, SciPy)
Status: Implementable Engineering Protocol

1. Deep-Dive: Core Technical Hurdles

Translating an epistemic discovery architecture into functional software requires addressing five fundamental computational and mathematical barriers.

1.1 Combinatorial Explosion in Adversarial Deliberation

- **The Challenge:** Generating and falsifying candidate hypotheses ($H_1, \dots, H_n$) across continuous parameter domains and non-linear functional forms produces an intractable search tree ($\mathcal{O}(B^d)$).
- **Failure Mode:** Exhaustion of compute budgets on algebraically trivial variations, or premature convergence to local minima.
- **Engineering Mitigation:** Constrain symbolic generation with grammar-guided genetic programming (e.g., PySR/SymbolicRegression) bounded by dimensional invariants.

1.2 The Model Discrepancy vs. Sensor Noise Confounder

- **The Challenge:** In the observation decomposition:
$$y = f_{\text{model}}(x) + \delta_{\text{model}}(x) + \epsilon_{\text{sensor}}$$
separating $delta_textmodel(x)$ (unmodeled physics) from $epsilon_textsensor$ (uncalibrated systematic drift) is fundamentally underdetermined without an absolute reference standard.
- **Failure Mode:** Machine learning models overfit $\delta_{\text{model}}(x)$, labeling sensor miscalibration or thermal drift as "new physics."
- **Engineering Mitigation:** Require multi-instrument orthogonal measurement diversity where no two sensors share the same physical measurement mechanism.

1.3 Intractability of Automated Invariant & Symmetry Discovery

- **The Challenge:** Finding continuous Lie symmetries and conserved currents for arbitrary non-linear PDEs requires solving determining equations via differential Gröbner bases, which carry **double-exponential complexity**.

- **Failure Mode:** Solvers stall indefinitely or run out of memory on coupled multi-field equations.
- **Engineering Mitigation:** Hybridize symbolic methods with numerical invariant testing (e.g., assessing conservation violation residuals $\sigma = \frac{\partial \rho}{\partial t} + \nabla \cdot \mathbf{J}$ along sampled trajectory manifolds).

1.4 General *A Posteriori* Numerical Error Bounds

- **The Challenge:** Distinguishing numerical discretization artifacts from physical discrepancies ($\|u - u_h\| \leq \eta(u_h, f)$) lacks sharp, computable bounds for non-linear hyperbolic systems and turbulent flows.
- **Failure Mode:** Overly conservative bounds invalidate genuine predictions; loose bounds permit numerical artifacts to be promoted as discoveries.
- **Engineering Mitigation:** Restrict Phase 0/1 claims to elliptic and parabolic regimes with established Dual Weighted Residual (DWR) estimators before scaling to chaotic regimes.

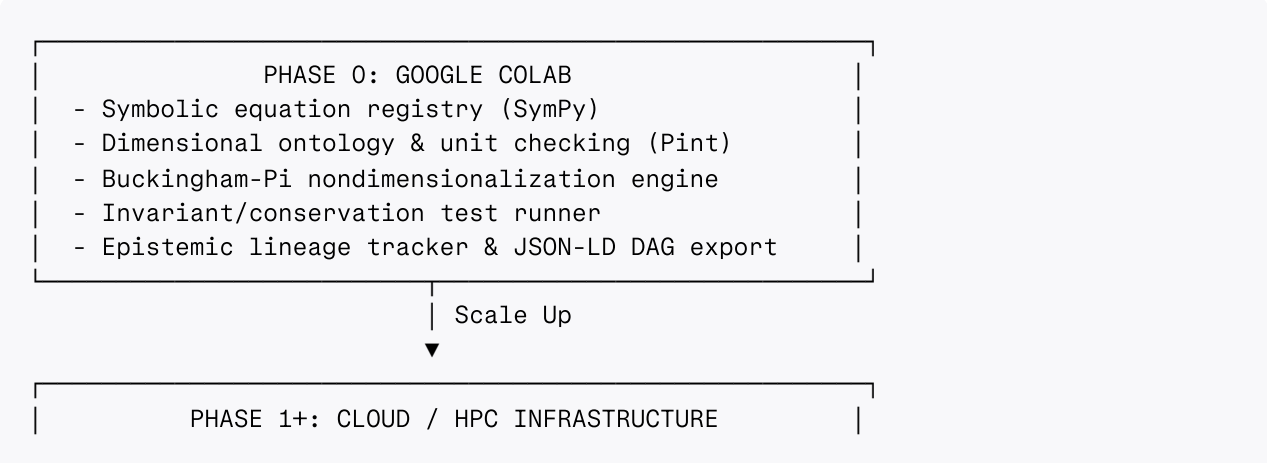
1.5 Epistemic Graph vs. Tensor Streaming Impedance Mismatch

- **The Challenge:** Combining high-bandwidth tensor streams (GB/s) with deeply nested recursive lineage traversals (`RootAssumptions(C)`) degrades performance across standard graph engines.
- **Failure Mode:** Bottlenecks where lineage recording takes significantly longer than the actual scientific simulation.
- **Engineering Mitigation:** Separate storage tiers: store high-volume numerical arrays in columnar Apache Parquet/Arrow, storing only content-addressed cryptographic hashes and edge metadata in the semantic lineage graph.

2. Feasibility of Google Colab as a Phase 0 Sandbox

2.1 Colab Viability Verdict: YES — Highly Recommended for Phase 0

Google Colab provides an optimal, zero-cost sandbox to prototype the foundational kernel before deploying high-cost distributed cloud infrastructure.



```
|  - Zero-copy Apache Arrow Flight data plane           |
|  - Distributed PDE / finite-element solvers           |
|  - Multi-agent adversarial deliberation tournaments    |
|  - Physical laboratory robotics & hardware-in-the-loop|
└──────────────────────────────────────────────────────┘
```

2.2 What Colab Can and Cannot Do

Capability	In Scope for Colab (Phase 0)	Requires HPC / Cloud Cluster (Phase 1+)
Dimensional Validation Engine	**Yes** (Instant execution)	—
Buckingham-π Reduction	**Yes** (SymPy matrix rank)	—
Epistemic Type Graph Tracking	**Yes** (NetworkX / JSON-LD)	Graph triplestore scaling
1D / 2D ODE/PDE Solvers	**Yes** (SciPy / JAX on T4 GPU)	Multi-node 3D Navier-Stokes
Multi-Agent Deliberation	**Yes** (Toy prototype: 2–4 models)	Large-scale parallel tournaments
Physical Sensor Ingestion	Synthetic data only	Real-time gRPC telemetry streams

3. Working Google Colab Starter Implementation

Below is a self-contained Python script designed to run directly in a Google Colab notebook cell. It implements the core primitives:

1. **Dimensional Validation & Invariant Checking**
2. **Automated Buckingham-π Nondimensionalization**
3. **Epistemic Lineage Graph with `ASSUMED_BY` Querying**

```
# ======================================================================
# APEDP PHASE 0 KERNEL PROTOTYPE FOR GOOGLE COLAB
# Install dependencies: !pip install pint sympy networkx
# ======================================================================

import pint
import sympy as sp
import networkx as nx
from typing import Dict, List, Any

# ----------------------------------------------------------------------
# 1. DIMENSIONAL ONTOLOGY ENGINE
# ----------------------------------------------------------------------
ureg = pint.UnitRegistry()

class DimensionalGuardrail:
    """Enforces dimensional homogeneity and catches illegal expressions."""

    @staticmethod
    def validate_decay_exponent(exponent_str: str) -> bool:
        """Rule: Exponents in functions like exp(-x) MUST be dimensionless."""
        try:
```

```
            qty = ureg(exponent_str)
            if qty.dimensionless:
                return True
            else:
                print(f"[REJECTED] Inhomogeneous exponent: {exponent_str} has dimens
                return False
        except Exception as e:
            print(f"[ERROR] Parsing failed for {exponent_str}: {e}")
            return False

# ---------------------------------------------------------------------
# 2. BUCKINGHAM-PI NONDIMENSIONALIZATION ENGINE
# ---------------------------------------------------------------------
class BuckinghamPiEngine:
    """Computes dimensionless Pi groups from a system of variables."""

    @staticmethod
    def compute_pi_groups(variables: Dict[str, Dict[str, int]]):
        """
        variables: dict of var_name -> {'M': a, 'L': b, 'T': c}
        Returns nullspace basis vectors representing Pi terms.
        """
        var_names = list(variables.keys())
        dims = ['M', 'L', 'T']

        # Build dimensional matrix (rows = fundamental dimensions, cols = variables)
        dim_matrix = []
        for d in dims:
            row = [variables[v].get(d, 0) for v in var_names]
            dim_matrix.append(row)

        M = sp.Matrix(dim_matrix)
        rank = M.rank()
        num_vars = len(var_names)
        num_pi = num_vars - rank

        print(f"[Buckingham-Pi] Variables: {num_vars}, Rank: {rank} -> {num_pi} Dime

        nullspace = M.nullspace()
        pi_groups = []
        for i, vec in enumerate(nullspace):
            terms = [f"{var_names[j]}^({vec[j]})" for j in range(num_vars) if vec[j]
            pi_expr = " * ".join(terms)
            pi_groups.append(f"Pi_{i+1} = {pi_expr}")

        return pi_groups

# ---------------------------------------------------------------------
# 3. EPISTEMIC LINEAGE GRAPH ENGINE
# ---------------------------------------------------------------------
class EpistemicGraph:
    """Knowledge DAG enforcing typed scientific entities and functional edges."""

    def __init__(self):
        self.graph = nx.DiGraph()
```

```
    def add_node(self, node_id: str, node_type: str, metadata: Dict[str, Any]):
        valid_types = [
            "Observation", "Measurand", "State", "Equation",
            "Model", "Hypothesis", "Prediction", "Evidence", "Claim"
        ]
        if node_type not in valid_types:
            raise ValueError(f"Invalid Epistemic Type: {node_type}")
        self.graph.add_node(node_id, entity_type=node_type, **metadata)

    def add_lineage_edge(self, source_id: str, target_id: str, edge_type: str):
        valid_edges = [
            "DERIVED_FROM", "INFERRED_BY", "ASSUMED_BY",
            "MEASURED_BY", "SIMULATED_BY", "APPROXIMATED_FROM", "CONTRADICTED_BY"
        ]
        if edge_type not in valid_edges:
            raise ValueError(f"Invalid Lineage Edge: {edge_type}")
        self.graph.add_edge(source_id, target_id, edge_type=edge_type)

    def get_root_assumptions(self, claim_id: str) -> List[str]:
        """Finds all upstream nodes linked via ASSUMED_BY."""
        ancestors = nx.ancestors(self.graph, claim_id)
        assumptions = []
        for ancestor in ancestors:
            for _, target, data in self.graph.out_edges(ancestor, data=True):
                if data.get('edge_type') == 'ASSUMED_BY':
                    assumptions.append(ancestor)
        return list(set(assumptions))

# ---------------------------------------------------------------------
# 4. EXECUTION DEMONSTRATION
# ---------------------------------------------------------------------
if __name__ == "__main__":
    print("=== STEP 1: DIMENSIONAL GUARDRAIL CHECK ===")
    guard = DimensionalGuardrail()
    guard.validate_decay_exponent("5 * joule")       # Should FAIL
    guard.validate_decay_exponent("2.5 * meter / (5 * meter)") # Should PASS

    print("\n=== STEP 2: BUCKINGHAM-PI NONDIMENSIONALIZATION ===")
    # Simple pendulum: length (L), gravity (L/T^2), mass (M), period (T)
    pendulum_vars = {
        'm': {'M': 1, 'L': 0, 'T': 0},
        'l': {'M': 0, 'L': 1, 'T': 0},
        'g': {'M': 0, 'L': 1, 'T': -2},
        't': {'M': 0, 'L': 0, 'T': 1}
    }
    pi_terms = BuckinghamPiEngine.compute_pi_groups(pendulum_vars)
    for p in pi_terms:
        print(" ", p)

    print("\n=== STEP 3: EPISTEMIC LINEAGE GRAPH ===")
    eg = EpistemicGraph()

    # Register lineage chain
    eg.add_node("Axiom_01", "Hypothesis", {"desc": "Ideal frictionless pivot"})
    eg.add_node("Eq_01", "Equation", {"latex": "T = 2*pi*sqrt(l/g)"})
    eg.add_node("Pred_01", "Prediction", {"period_sec": 2.006})
```

```
    eg.add_node("Claim_01", "Claim", {"statement": "Period scales strictly with sqrt

    eg.add_lineage_edge("Axiom_01", "Eq_01", "ASSUMED_BY")
    eg.add_lineage_edge("Eq_01", "Pred_01", "DERIVED_FROM")
    eg.add_lineage_edge("Pred_01", "Claim_01", "DERIVED_FROM")

    roots = eg.get_root_assumptions("Claim_01")
    print(f"Root unverified assumptions for 'Claim_01': {roots}")
```